

Research Notes on Byte pair encoding tokenization

Byte pair encoding tokenization

>> Section 1

Example Suppose we are encoding the previous example of "aaabdaabac", with a specified vocabulary size of 6, then it would first be encoded as "0, 0, 0, 1, 2, 0, 0, 0, 1, 0, 3" with a vocabulary of "a=0, b=1, d=2, c=3". Then it would proceed as before, and obtain "4, 5, 2, 4, 5, 0, 3" with a vocabulary of "a=0, b=1, d=2, c=3, aa=4, ab=5". So far this is essentially the same as before. However, if we only had specified a vocabulary size of 5, then the process would stop at vocabulary "a=0, b=1, d=2, c=3, aa=4", so that the example would be encoded as "4, 0, 1, 2, 4, 0, 1, 0, 3". Conversely, if we had specified a vocabulary size of 8, then it would be encoded as "7, 6, 0, 3", with a vocabulary of "a=0, b=1, d=2, c=3, aa=4, ab=5, aaab=6, aaabd=7". This is not maximally compressed, because modified BPE does not aim for maximum compression. Instead, it aims for an encoding that is efficient and practical for language model training.

>> Section 2

Original algorithm The original BPE algorithm operates by iteratively replacing the most common contiguous sequences of characters in a target text with unused 'placeholder' bytes. The iteration ends when no sequences can be found, leaving the target text effectively compressed. Decompression can be performed by reversing this process, querying known placeholder terms against their corresponding denoted sequence, using a lookup table. In the original paper, this lookup table is encoded and stored alongside the compressed text.

>> Section 3

Modified algorithm The original BPE algorithm is modified for use in language modeling, especially for large language models based on neural networks. Compared to the original BPE, the modified BPE does not aim to maximally compress text, but rather, to encode plaintext into "tokens", which are natural numbers. All the unique tokens found in a corpus are listed in a token vocabulary. The token vocabulary can also include some other special tokens, relative to the use case. The size of the token vocabulary, in the case of GPT-3.5 and GPT-4, is 100258 (100000 from BPE algorithm and 258 included as special tokens). The modified tokenization algorithm initially treats the set of unique characters as 1-character-long n-grams (the initial tokens). Then, successively, the most frequent pair of adjacent tokens is merged into a new, longer n-gram and all instances of the pair are replaced by this new token. This is repeated until a vocabulary of prescribed size is obtained. Note that new words can always be constructed from final vocabulary tokens and initial-set characters. This modified BPE approach has been

extended from spoken language to sign language in recent years.

>> Section 4

The byte pair "aa" occurs most often, so it will be replaced by a byte that is not used in the data, such as "Z". Now there is the following data and replacement table:

>> Section 5

Byte-level BPE In the above example, the output of the BPE is a vocabulary, which can be used to encode any text that is written with the letters "abcd". It will not be able to encode text containing other symbols, such as "no". Even giving each of the 26 letters an entry in the vocabulary, since there are many languages in the world using many different scripts, inevitably some symbols would be unencodable by such a vocabulary. One solution is to replace any unencodable symbol with a special symbol named UNK ("unknown"). The byte-level BPE is another approach. It simply converts the text into UTF-8 first, and treat it as a stream of bytes. This guarantees that any text encoded in UTF-8 can be encoded by the BPE. This has been used in BERT-like models like RoBERTa, BART, and DeBERTa, and GPT-like models like GPT-2.

>> Section 6

This data cannot be compressed further by byte-pair encoding because there are no pairs of bytes that occur more than once. To decompress the data, simply perform the replacements in the reverse order.

>> Section 7

In computing, byte-pair encoding (BPE), or digram coding, is an algorithm, first described in 1994 by Philip Gage, for encoding strings of text into smaller strings by creating and using a translation table. A slightly modified version of the algorithm is used in large language model tokenizers. The original version of the algorithm focused on compression. It replaces the highest-frequency pair of bytes with a new byte that was not contained in the initial dataset. A lookup table of the replacements is required to rebuild the initial dataset. The modified version builds "tokens" (units of recognition) that match varying amounts of source text, from single characters (including single digits or single punctuation marks) to whole words (even long compound words).